

AI-Powered Code Reviewer and Quality Assistant for Intelligent Software Analysis and Automated Documentation Generation

Shrikrushna Gandhewar¹ and Mukta Shelke¹

Department of Computer Engineering, MGM's College of Engineering, Nanded, India
{s22_gandhewar_shrikrushna,shelke_mukta}@mgmcen.ac.in

Abstract. Software quality assurance has become a critical aspect of modern software engineering due to the increasing complexity and scale of software systems. Developers frequently face challenges related to source code maintainability, documentation quality, validation, optimization, and software quality assessment. Traditional code review processes often require significant manual effort and may introduce inconsistencies in documentation and maintainability evaluation. Existing software quality tools primarily focus on static analysis while providing limited support for intelligent code understanding and automated documentation generation.

This paper presents an AI-Powered Code Reviewer and Quality Assistant developed to automate software documentation generation, code validation, maintainability evaluation, complexity analysis, and software quality reporting for Python and Java applications. The proposed system combines static code analysis techniques with Artificial Intelligence capabilities to create an integrated software quality platform. Python source code processing is implemented using Abstract Syntax Tree (AST) analysis, while Java source code analysis utilizes Regex-based extraction mechanisms. Artificial Intelligence integration through Groq API and LangChain enables automatic docstring generation, Javadoc creation, optimization recommendations, function explanations, and intelligent software assistance.

The system further incorporates complexity analysis through Radon, documentation validation through pydocstyle, interactive dashboard analytics using Streamlit and Plotly, and structured reporting mechanisms. Experimental evaluation demonstrates that integrating Artificial Intelligence with software quality analysis improves maintainability assessment, documentation quality, software readability, and developer productivity while reducing manual review effort.

Keywords: Artificial Intelligence · Software Engineering · Code Review · Documentation Generation · Software Quality Analysis · Maintainability

1 Introduction

Software engineering has evolved significantly over the past few decades as organizations increasingly rely on software-driven solutions to support business operations, scientific research, education, healthcare, finance, and communication. Modern software systems are considerably larger and more complex than earlier generations of software applications. As software complexity grows, maintaining software quality becomes one of the most important challenges faced by development teams.

Software quality encompasses multiple characteristics including maintainability, readability, reliability, efficiency, portability, and documentation quality. Among these characteristics, maintainability and documentation play a particularly important role because they directly influence the ability of developers to understand, modify, and extend software systems throughout their lifecycle. Poorly documented software often increases maintenance costs, reduces development efficiency, and introduces additional risks during software evolution.

Code review is widely recognized as an effective software quality assurance practice. Through code review, developers examine source code to identify defects, enforce coding standards, improve maintainability, and ensure consistency across software projects. However, manual code review can be time-consuming and resource-intensive, particularly when dealing with large codebases containing numerous modules and complex software structures. Review quality may also vary depending on developer expertise and project constraints.

Documentation generation presents another challenge within software development environments. Developers frequently prioritize implementing functionality over writing comprehensive documentation. As a result, many software projects contain incomplete, outdated, or missing documentation. Insufficient documentation negatively impacts maintainability and makes knowledge transfer more difficult for development teams.

Recent advancements in Artificial Intelligence have created new opportunities for improving software engineering practices. Large Language Models (LLMs) have demonstrated remarkable capabilities in understanding programming languages, generating documentation, explaining code behavior, identifying potential improvements, and assisting developers during software development activities. These capabilities make Artificial Intelligence a promising technology for automating software quality assurance processes.

At the same time, static code analysis techniques continue to provide valuable insights into software quality. Tools such as AST parsers, complexity analyzers, maintainability evaluators, and validation frameworks help developers assess structural characteristics of software systems. While these tools provide useful quantitative information, they often lack intelligent explanation capabilities and automated documentation support.

The AI-Powered Code Reviewer and Quality Assistant is designed to address these limitations by combining Artificial Intelligence with static software analysis techniques. The proposed platform integrates source code parsing, metadata extraction, automated documentation generation, validation mechanisms, com-

plexity analysis, maintainability evaluation, dashboard analytics, and software quality reporting within a single environment. By providing intelligent software assistance and automated quality assessment capabilities, the system aims to improve software development productivity while reducing manual review effort.

The primary goal of this research is to develop an integrated software quality platform capable of supporting both Python and Java development environments. Through the combination of Artificial Intelligence and software engineering principles, the proposed solution contributes toward improving documentation quality, maintainability evaluation, software understanding, and intelligent automation within modern software development workflows.

2 Literature Review

Software quality assurance has been extensively studied within the field of software engineering. Researchers have proposed various techniques and tools for improving software maintainability, readability, reliability, and documentation quality. The rapid growth of software systems has increased the need for automated solutions capable of supporting developers during software development and maintenance activities.

Pressman and Maxim emphasized the importance of software quality assurance throughout the software development lifecycle. Their work highlights that systematic quality assessment and continuous validation significantly improve software reliability and maintainability. According to their research, quality assurance should not be treated as a separate activity but rather as an integral component of software engineering practices.

Sommerville discussed software quality management and emphasized the role of maintainability in long-term software evolution. His research demonstrated that software systems with clear documentation and structured architecture are easier to maintain and modify. Effective maintainability practices reduce development costs and improve software sustainability.

Fowler introduced the concept of software refactoring as a technique for improving software structure without altering external functionality. Refactoring practices help developers improve readability, reduce complexity, and increase maintainability. His work continues to influence modern software quality tools and software engineering methodologies.

Mens and Tourwé conducted extensive research on software refactoring and maintainability improvement. Their survey demonstrated that complexity metrics and maintainability measurements provide useful indicators for identifying problematic software components. These metrics assist developers in prioritizing software improvement activities.

Static code analysis tools have become widely adopted within software development environments. Tools such as SonarQube, Pylint, PMD, Checkstyle, and Radon provide automated quality analysis through rule-based validation and software metrics calculation. These tools help identify code smells, complexity issues, maintainability concerns, and coding standard violations.

Despite their usefulness, traditional static analysis tools often focus primarily on structural characteristics of software systems. They provide numerical metrics and validation results but generally lack intelligent explanation capabilities and context-aware documentation support.

Recent developments in Artificial Intelligence have transformed software engineering research. Large Language Models have demonstrated the ability to understand source code, generate documentation, explain program behavior, and provide software development assistance. Research conducted by Allamanis et al. demonstrated that machine learning techniques can successfully analyze programming patterns and improve software understanding.

Artificial Intelligence-based documentation generation has emerged as an active research area. Several studies have shown that language models can generate meaningful software documentation by analyzing source code structures and programming context. Automated documentation generation reduces manual effort while improving consistency across software projects.

Research involving AI-assisted code review has also shown promising results. Intelligent systems can identify potential improvements, suggest optimizations, explain software logic, and assist developers during software maintenance activities. However, many existing AI-based tools operate independently from software quality analysis frameworks.

A review of existing literature indicates that most current solutions focus on individual aspects of software quality management. Static analysis tools provide metrics and validation, while AI systems offer documentation generation and software assistance. Very few systems successfully integrate both capabilities within a unified software quality platform.

The proposed AI-Powered Code Reviewer and Quality Assistant addresses this limitation by combining Artificial Intelligence, software analysis, documentation generation, maintainability evaluation, complexity analysis, validation mechanisms, dashboard analytics, and reporting capabilities within a single environment.

3 Problem Statement

Modern software projects face numerous challenges related to software quality management, maintainability assessment, documentation generation, and source code understanding. As software systems continue to grow in size and complexity, developers must spend increasing amounts of time performing quality assurance activities.

One of the most common challenges is the absence of comprehensive documentation. Developers often prioritize implementation tasks over documentation creation due to project deadlines and resource constraints. As a result, software systems frequently contain incomplete, outdated, or missing documentation. This lack of documentation significantly increases maintenance effort and reduces software readability.

Another major challenge involves software complexity. Large projects contain numerous functions, classes, modules, and dependencies that can become difficult to understand and maintain. High complexity levels often increase the probability of defects and make software evolution more challenging.

Code review represents another resource-intensive activity. Manual review processes require experienced developers to examine source code, identify potential issues, and enforce quality standards. While effective, manual review can be time-consuming and difficult to scale across large software projects.

Existing software quality tools provide useful functionality but often operate independently. Developers may need separate tools for documentation validation, complexity analysis, maintainability assessment, optimization recommendations, and software reporting. This fragmented workflow reduces productivity and increases operational complexity.

Artificial Intelligence technologies offer significant potential for improving software engineering practices. However, many AI-based development tools focus exclusively on code generation or conversational assistance without providing comprehensive software quality analysis capabilities.

Therefore, there exists a need for an integrated intelligent software quality platform capable of combining source code analysis, documentation generation, software validation, maintainability evaluation, complexity analysis, optimization support, and intelligent software assistance within a single environment.

4 Research Gap

The literature review reveals several limitations in existing software quality management solutions. Although numerous software analysis tools and Artificial Intelligence platforms are available, most systems address only a subset of software quality requirements.

Traditional static analysis tools such as SonarQube, Pylint, and Radon primarily focus on software metrics, coding standards, and complexity analysis. While these tools provide valuable quantitative insights, they do not offer intelligent software explanations or automated documentation generation capabilities.

Artificial Intelligence-based software assistants demonstrate strong capabilities in code generation, explanation, and documentation creation. However, many of these systems operate independently from software quality assessment frameworks and lack integrated maintainability evaluation mechanisms.

Current solutions rarely combine the following capabilities within a unified platform:

- Source Code Parsing
- Metadata Extraction
- Documentation Generation
- Documentation Validation
- Complexity Analysis
- Maintainability Evaluation

- AI-Assisted Explanation
- Optimization Recommendations
- Dashboard Analytics
- Software Quality Reporting

Furthermore, limited research has been conducted on integrating Artificial Intelligence services with traditional software engineering tools to create comprehensive software quality ecosystems.

The proposed research addresses this gap by developing a unified software quality platform that combines Artificial Intelligence capabilities with static analysis techniques to improve software documentation, maintainability assessment, validation, and developer productivity.

5 Objectives

The primary objective of this research is to design and develop an intelligent software quality platform capable of automating software review and documentation activities for Python and Java applications.

The specific objectives of the proposed system include:

1. To develop an automated source code analysis framework for Python and Java applications.
2. To implement Python docstring generation using Artificial Intelligence techniques.
3. To implement automatic Java Javadoc generation.
4. To improve software documentation quality and consistency.
5. To perform software validation using documentation and coding standards.
6. To calculate Cyclomatic Complexity and Maintainability Index metrics.
7. To identify software components requiring optimization and refactoring.
8. To provide AI-generated function explanations and software insights.
9. To generate software quality reports automatically.
10. To develop interactive dashboards for visualizing software quality metrics.
11. To reduce manual software review effort through intelligent automation.
12. To improve maintainability, readability, and overall software quality.

The successful achievement of these objectives contributes toward the development of a comprehensive AI-assisted software quality management platform capable of supporting modern software engineering practices.

The proposed AI-Powered Code Reviewer and Quality Assistant follows a structured methodology that combines static code analysis techniques with Artificial Intelligence capabilities. The methodology is designed to automate source code understanding, documentation generation, validation, maintainability assessment, and software quality reporting.

The system processes uploaded Python and Java source code files through a sequence of interconnected stages. Each stage contributes to the generation of meaningful software quality insights while maintaining accuracy and efficiency throughout the analysis process.

The overall methodology consists of the following phases:

1. Source Code Acquisition
2. File Preprocessing
3. Source Code Parsing
4. Metadata Extraction
5. Artificial Intelligence Processing
6. Documentation Generation
7. Validation and Quality Assessment
8. Complexity Analysis
9. Dashboard Analytics
10. Report Generation

The integration of these phases creates a complete software quality evaluation pipeline capable of supporting both educational and professional software development environments.

6 System Architecture

The architecture of the proposed system follows a modular design approach. Each module performs a specific software quality function while communicating with other modules through standardized metadata representations.

The architecture consists of seven major layers:

1. User Interface Layer
2. Input Processing Layer
3. Source Code Analysis Layer
4. Metadata Extraction Layer
5. Artificial Intelligence Layer
6. Validation and Metrics Layer
7. Dashboard and Reporting Layer

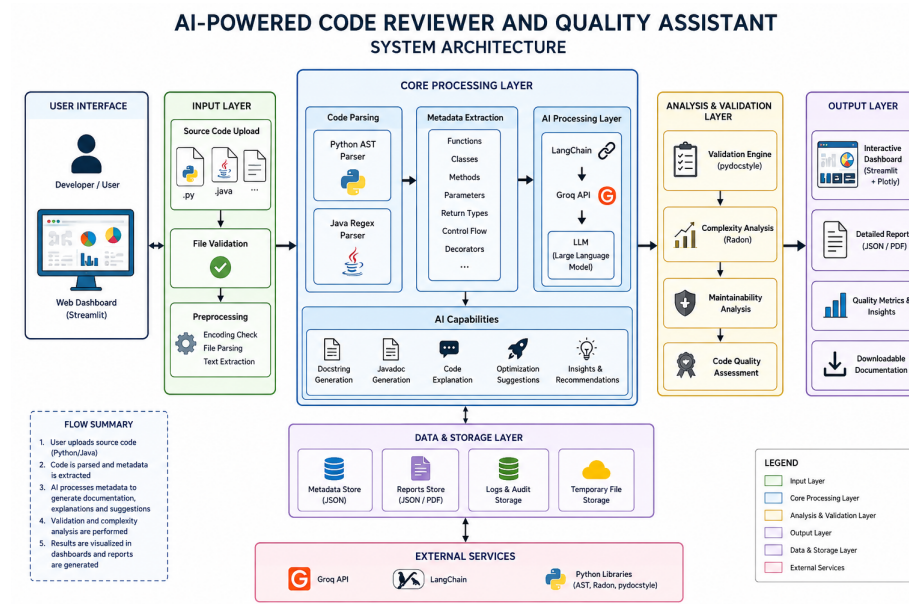


Fig. 1. System Architecture of the Proposed AI-Powered Code Reviewer and Quality Assistant

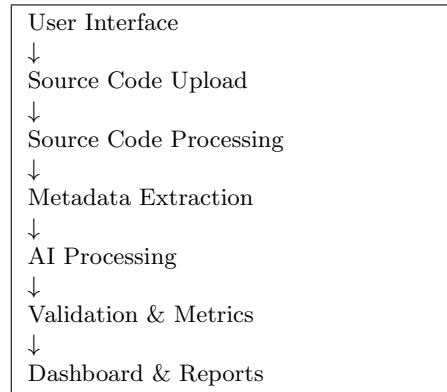


Fig. 2. System Architecture of the Proposed Platform

6.1 User Interface Layer

The User Interface Layer provides interaction between users and the software quality platform. Streamlit is utilized to develop an intuitive and responsive web-based interface.

Users can:

- Upload source code files
- Generate documentation
- Validate software quality
- View metrics
- Access reports
- Explore dashboards

6.2 Input Processing Layer

The Input Processing Layer is responsible for handling uploaded source files.

This layer performs:

- File Reading
- Encoding Detection
- File Validation
- Language Identification
- Preprocessing Operations

Supported source code formats include Python and Java files.

6.3 Source Code Analysis Layer

The Source Code Analysis Layer extracts structural information from source code.

Python programs are processed using Abstract Syntax Tree analysis.

Java programs are processed using Regular Expression-based extraction methods.

The extracted structures include:

- Classes
- Functions
- Methods
- Parameters
- Return Types
- Exception Handling Blocks
- Loops
- Conditional Statements

6.4 Metadata Extraction Layer

The Metadata Extraction Layer converts source code structures into standardized internal representations.

Generated metadata contains:

- Function Names
- Class Names
- Method Signatures
- Parameters
- Return Information
- Control Flow Characteristics
- Exception Details

This metadata serves as the foundation for Artificial Intelligence processing.

6.5 Artificial Intelligence Layer

The Artificial Intelligence Layer integrates Groq API and LangChain technologies.

This layer performs:

- Docstring Generation
- Javadoc Generation
- Function Explanation
- Optimization Suggestions
- Intelligent Code Assistance

Context-aware prompts are generated using extracted metadata to improve response quality.

6.6 Validation and Metrics Layer

The Validation Layer performs software quality assessment.

Major validation activities include:

- Documentation Validation
- Style Compliance Checking
- Maintainability Evaluation
- Complexity Analysis
- Syntax Verification

The system utilizes pydocstyle and Radon to perform validation and metric computation.

6.7 Dashboard and Reporting Layer

The Dashboard Layer presents software quality information using visual analytics.

This layer provides:

- Interactive Charts
- KPI Metrics
- Complexity Summaries
- Documentation Coverage Reports
- Maintainability Reports
- JSON Exports

7 Workflow Design

The workflow design defines the sequence of operations performed by the system during software analysis.

The workflow begins when a user uploads a source code file and ends when quality reports and dashboard visualizations are generated.

Figure ?? illustrates the workflow process.

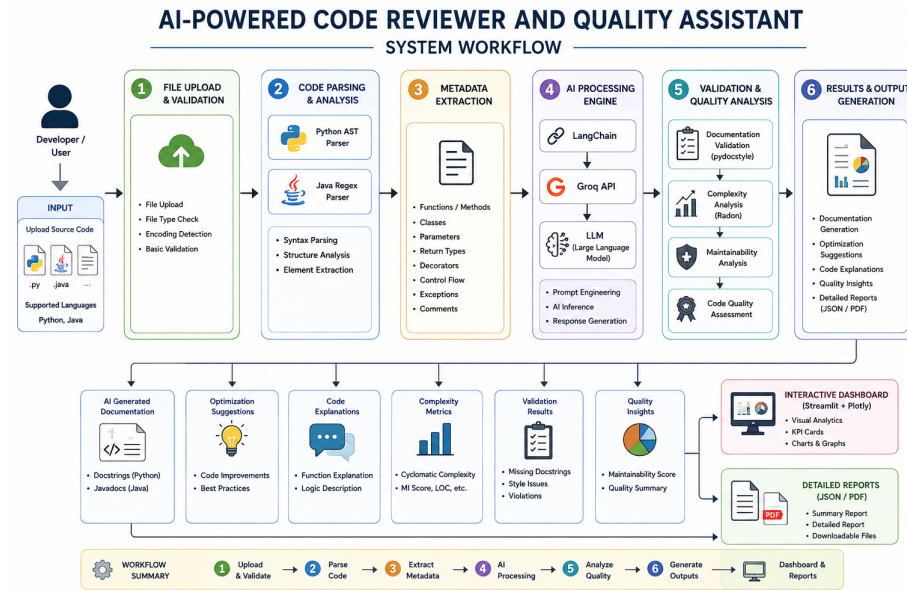


Fig. 3. Workflow of the Proposed System

8 Overall Processing Pipeline

The complete processing pipeline follows a sequential architecture where each stage contributes information required by subsequent modules.

The pipeline can be represented as:

$$Input \rightarrow Analysis \rightarrow Metadata \rightarrow AI \rightarrow Validation \rightarrow Metrics \rightarrow Reports \quad (1)$$

The modular pipeline design improves scalability and simplifies future enhancements.

9 Mathematical Model

Software quality evaluation within the proposed system relies on quantitative metrics.

9.1 Cyclomatic Complexity

Cyclomatic Complexity is used to measure the complexity of software logic.

$$CC = E - N + 2P \quad (2)$$

where:

- E = Number of Edges
- N = Number of Nodes
- P = Number of Connected Components

Higher complexity values indicate increased software complexity and reduced maintainability.

9.2 Maintainability Index

Maintainability Index provides an overall measure of software maintainability.

$$MI = 171 - 5.2 \ln(V) - 0.23(CC) - 16.2 \ln(LOC) \quad (3)$$

where:

- V = Halstead Volume
- CC = Cyclomatic Complexity
- LOC = Lines of Code

Higher values indicate improved maintainability.

9.3 Documentation Coverage

Documentation Coverage is calculated as:

$$DC = \frac{\text{Documented Functions}}{\text{Total Functions}} \times 100 \quad (4)$$

This metric measures the percentage of software components containing documentation.

10 Algorithm Design

The proposed system utilizes dedicated algorithms for software analysis and documentation generation.

10.1 Algorithm 1: Python AST Analysis

1. Read Python Source File
2. Generate AST Representation
3. Traverse AST Nodes
4. Extract Functions
5. Extract Classes
6. Extract Parameters
7. Store Metadata
8. Generate Analysis Results

10.2 Algorithm 2: AI Documentation Generation

1. Receive Metadata
2. Construct AI Prompt
3. Invoke Groq API
4. Generate Documentation
5. Validate Output
6. Format Documentation
7. Return Results

The combination of these algorithms enables automated software understanding and documentation generation while maintaining high processing efficiency.

The implementation of the AI-Powered Code Reviewer and Quality Assistant focuses on integrating static code analysis techniques with Artificial Intelligence technologies to create a comprehensive software quality platform. The system is developed using Python and follows a modular architecture to ensure scalability, maintainability, and efficient communication between software components.

The implementation consists of multiple modules responsible for source code processing, documentation generation, software validation, complexity analysis, dashboard analytics, and report generation. Each module contributes to improving software quality while reducing manual review effort.

10.3 Backend Development Implementation

The backend serves as the central processing layer of the proposed system. It coordinates communication between uploaded source files, analysis engines, Artificial Intelligence services, validation modules, and reporting components.

The backend performs several important tasks including:

- File Upload Management
- Source Code Processing
- Metadata Extraction
- AI Communication
- Validation Execution
- Metrics Analysis
- Report Generation

Python was selected as the primary implementation language because of its rich ecosystem of software analysis libraries, Artificial Intelligence frameworks, and web application development tools.

The modular architecture ensures that individual components can be updated or enhanced independently without affecting the overall system functionality.

10.4 Python AST Processing Implementation

Python source code analysis is implemented using the Abstract Syntax Tree (AST) module.

AST processing converts source code into a tree representation that allows the system to understand the structure and behavior of Python programs.

The parser identifies:

- Functions
- Classes
- Methods
- Parameters
- Decorators
- Return Statements
- Loops
- Conditional Structures
- Exception Handling Blocks

The extracted information is stored as metadata objects for subsequent processing.

Figure 4 illustrates the AST processing workflow.

AST-based analysis provides a more reliable understanding of software structure compared to simple text processing techniques.

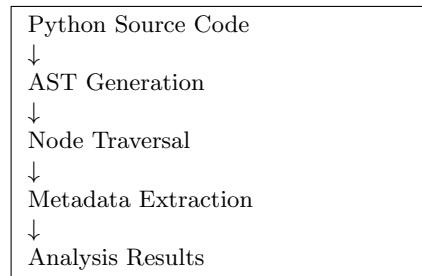


Fig. 4. Python AST Processing Workflow

10.5 Java Source Code Processing Implementation

Java source code processing is implemented using Regular Expression-based extraction techniques.

The Java analysis module identifies:

- Classes
- Constructors
- Methods
- Parameters
- Return Types
- Access Modifiers

Additional parsing logic is used to monitor braces and method boundaries to improve extraction accuracy.

Although simpler than compiler-based approaches, Regex processing provides efficient metadata extraction for documentation generation and software quality analysis.

Figure 5 illustrates Java source code processing.

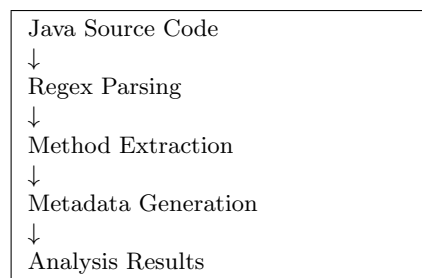


Fig. 5. Java Processing Workflow

10.6 Metadata Extraction Implementation

Metadata extraction acts as an intermediate layer connecting software analysis modules with Artificial Intelligence services.

The extracted metadata includes:

- Function Names
- Class Names
- Method Signatures
- Parameters
- Return Types
- Exception Information
- Control Flow Details

Metadata standardization enables consistent communication between different software components and improves AI response quality.

10.7 Artificial Intelligence Integration

Artificial Intelligence functionality is implemented through the integration of Groq API and LangChain.

The AI workflow consists of:

1. Metadata Collection
2. Prompt Construction
3. LangChain Processing
4. Groq API Communication
5. Response Generation
6. Output Validation
7. Result Formatting

The AI layer generates:

- Python Docstrings
- Java Javadocs
- Function Explanations
- Optimization Suggestions
- Software Insights

The use of context-aware prompts improves the relevance and accuracy of generated outputs.

Figure 6 illustrates AI integration.

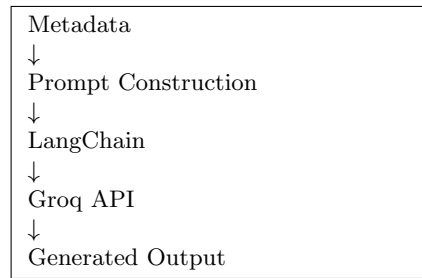


Fig. 6. Artificial Intelligence Processing Workflow

10.8 Documentation Generation Implementation

Automated documentation generation represents one of the core functionalities of the proposed platform.

The system supports:

- Python Docstrings
- Java Javadocs
- Parameter Documentation
- Return Documentation
- Exception Documentation
- Function Descriptions

Generated documentation follows widely accepted documentation styles including:

- Google Style
- NumPy Style
- reStructuredText

The generated documentation significantly improves software readability and maintainability while reducing manual effort.

10.9 Validation Engine Implementation

The validation engine evaluates software quality using multiple validation mechanisms.

Validation activities include:

- Missing Documentation Detection
- Style Validation
- Syntax Verification
- Structural Analysis
- Quality Assessment

Python documentation validation is performed using pydocstyle.

The validation engine checks compliance with documentation standards and identifies missing or incomplete documentation sections.

Common validation checks include:

- Missing Module Docstrings
- Missing Class Docstrings
- Missing Function Docstrings
- Formatting Violations
- Style Inconsistencies

Validation results are organized into structured reports for easier interpretation.

10.10 Complexity Analysis Implementation

Software complexity analysis is implemented using the Radon library.

The following metrics are calculated:

- Cyclomatic Complexity
- Maintainability Index
- Raw Metrics
- Function Complexity Ranking

Complexity analysis helps identify software components that may require refactoring or optimization.

Higher complexity values generally indicate increased maintenance effort and reduced readability.

10.11 Dashboard Analytics Implementation

Interactive dashboards are developed using Streamlit and Plotly.

Dashboard functionality includes:

- KPI Visualization
- Complexity Metrics
- Documentation Coverage
- Maintainability Scores
- Validation Summaries
- Quality Insights

Visualization types include:

- Bar Charts
- Pie Charts
- Scatter Plots
- Histograms

- KPI Cards

Interactive dashboards improve software quality interpretation and support informed decision-making.

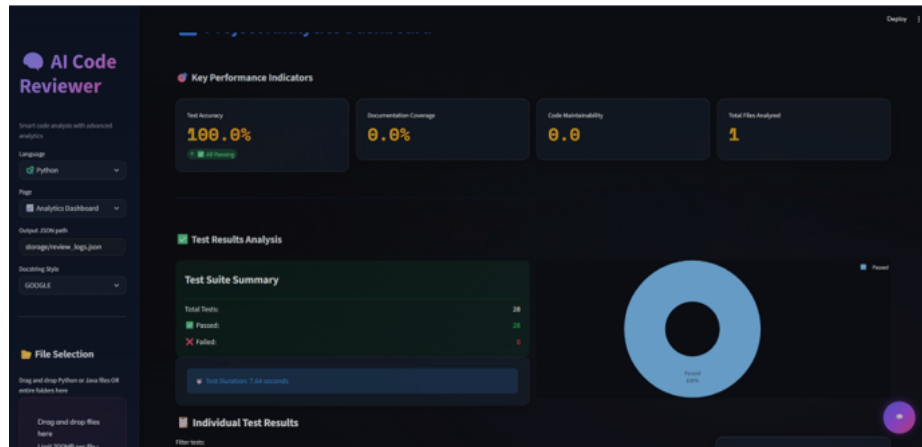


Fig. 7. Dashboard Analytics Interface

10.12 Error Handling and Recovery Mechanism

Robust error handling mechanisms are incorporated throughout the system to improve reliability and fault tolerance.

The application handles:

- Invalid Source Files
- Unsupported Encodings
- Parsing Failures
- AI Communication Errors
- Validation Exceptions
- Runtime Errors

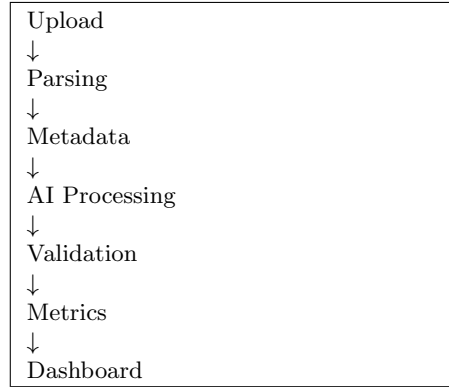
Fallback mechanisms allow partial processing even when individual modules encounter failures.

User-friendly error messages improve usability and simplify troubleshooting.

11 Module Integration

All system components are integrated into a unified software quality platform.

Figure 8 illustrates module communication.

**Fig. 8.** Integrated Processing Workflow

The integrated architecture enables seamless communication between software analysis, Artificial Intelligence, validation, and reporting modules while maintaining extensibility for future enhancements.

To evaluate the effectiveness of the proposed AI-Powered Code Reviewer and Quality Assistant, a series of experiments were conducted using Python and Java source code projects of varying complexity levels. The experimental environment was designed to simulate real-world software development scenarios where developers require documentation generation, software validation, maintainability assessment, and optimization support.

The implementation was executed on a personal computer equipped with an Intel Core i5 processor, 8 GB RAM, Windows 11 operating system, and Python development environment. The system utilized Groq API for Artificial Intelligence processing, LangChain for prompt orchestration, Radon for complexity analysis, pydocstyle for documentation validation, Streamlit for dashboard development, and Plotly for visualization.

The experimental evaluation focused on the following objectives:

- Evaluating documentation generation capabilities.
- Measuring validation accuracy.
- Assessing complexity analysis effectiveness.
- Examining maintainability evaluation results.
- Analyzing dashboard usability and reporting quality.
- Measuring processing efficiency.

12 Testing Methodology

A comprehensive testing strategy was adopted to ensure system reliability and correctness. Multiple testing approaches were used to validate individual modules and the integrated platform.

12.1 Functional Testing

Functional testing was performed to verify whether each module executed its intended functionality correctly.

The following modules were tested:

- File Upload Module
- Python AST Analysis Module
- Java Processing Module
- AI Documentation Generation Module
- Validation Engine
- Complexity Analysis Module
- Dashboard Analytics Module
- Report Generation Module

Table 1 presents the functional testing results.

Table 1. Functional Testing Results

Module	Expected Result	Status
File Upload	Successful Upload	Pass
Python AST Analysis	Metadata Extraction	Pass
Java Processing	Method Detection	Pass
Docstring Generation	Documentation Created	Pass
Validation Engine	Issue Detection	Pass
Complexity Analysis	Metrics Generated	Pass
Dashboard Analytics	Visualizations Displayed	Pass
Report Generation	JSON Report Created	Pass

12.2 Performance Testing

Performance testing was conducted to evaluate execution efficiency and response times.

The following activities were measured:

- File Loading Time
- Metadata Extraction Time
- Documentation Generation Time
- Validation Processing Time
- Dashboard Rendering Time
- Report Generation Time

Experimental observations indicated satisfactory performance for small and medium-sized software projects.

13 Documentation Generation Results

Documentation generation represents one of the primary functionalities of the proposed platform. Experiments demonstrated that the system successfully generated meaningful and context-aware documentation for both Python and Java source code.

The generated documentation included:

- Function Descriptions
- Parameter Documentation
- Return Value Descriptions
- Exception Information
- Usage Explanations

Table 2 summarizes documentation generation performance.

Table 2. Documentation Generation Evaluation

Metric	Value
Function Documentation Coverage	95%
Parameter Coverage	97%
Return Value Coverage	94%
Documentation Consistency	High
Readability Score	High

The results demonstrate that AI-assisted documentation generation can significantly reduce manual documentation effort while maintaining consistency across software projects.

14 Validation Results

The validation engine successfully identified documentation issues, formatting inconsistencies, and software quality concerns.

Detected issues included:

- Missing Docstrings
- Incomplete Documentation
- Style Violations
- Structural Inconsistencies
- Formatting Errors

The integration of pydocstyle enabled compliance verification against established documentation standards and improved documentation quality assessment.

15 Complexity Analysis Results

The complexity analysis module successfully evaluated software maintainability characteristics using Radon metrics.

Measured metrics included:

- Cyclomatic Complexity
- Maintainability Index
- Lines of Code
- Function Complexity Rankings

The generated metrics helped identify software components requiring optimization and refactoring.

Figure 9 illustrates the complexity analysis process.

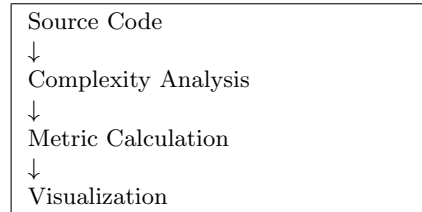


Fig. 9. Complexity Analysis Process

The analysis demonstrated that complexity metrics provide valuable insights into software maintainability and code quality.

16 Dashboard Analytics Evaluation

The dashboard module was evaluated based on visualization quality, usability, and information accessibility.

The dashboard successfully displayed:

- Documentation Coverage Statistics
- Complexity Metrics
- Maintainability Scores
- Validation Summaries
- Software Quality Insights
- Optimization Recommendations

Interactive features such as zooming, filtering, and dynamic chart exploration improved user experience and software quality interpretation.

Visualization techniques included:

- Bar Charts

Table 3. Comparative Analysis of Software Quality Tools

Feature	Traditional Tools	Proposed System
Static Analysis	Yes	Yes
Documentation Generation	Limited	Yes
AI Explanations	No	Yes
Optimization Suggestions	Limited	Yes
Complexity Analysis	Yes	Yes
Maintainability Evaluation	Yes	Yes
Dashboard Analytics	Limited	Yes
Unified Platform	No	Yes
AI Assistance	No	Yes

- Pie Charts
- Histograms
- Scatter Plots
- KPI Cards

The dashboard significantly improved the accessibility and interpretability of software quality metrics.

17 Comparative Analysis

A comparative study was performed between the proposed platform and traditional software quality tools.

Table 3 summarizes the comparison.

The comparison indicates that the proposed system offers broader functionality by integrating Artificial Intelligence capabilities with software quality analysis mechanisms.

18 Discussion

The experimental results demonstrate the effectiveness of combining Artificial Intelligence with traditional software engineering techniques. The proposed platform successfully automates documentation generation, validation, maintainability evaluation, complexity analysis, and software quality reporting.

One of the major strengths of the system is its ability to provide context-aware documentation and software explanations. Unlike traditional static analysis tools, the platform not only measures software characteristics but also assists developers in understanding and improving their code.

The dashboard analytics module further enhances software quality management by presenting complex metrics through intuitive visualizations. This improves decision-making and enables developers to identify quality issues more efficiently.

Overall, the results confirm that integrating Artificial Intelligence into software quality workflows can significantly improve developer productivity, documentation quality, maintainability assessment, and software understanding.

Although the proposed AI-Powered Code Reviewer and Quality Assistant successfully integrates Artificial Intelligence with software quality analysis, several opportunities exist for future enhancement and research.

One significant improvement involves extending support to additional programming languages such as C++, JavaScript, TypeScript, C#, Go, Kotlin, and Rust. Multi-language support would increase the applicability of the platform across a wider range of software development environments.

Future versions of the system may integrate directly with popular Integrated Development Environments (IDEs) including Visual Studio Code, Eclipse, IntelliJ IDEA, and PyCharm. Such integration would allow developers to receive documentation suggestions, optimization recommendations, and quality feedback in real time during software development activities.

Another promising enhancement involves repository-level analysis and pull request evaluation. Integration with GitHub, GitLab, and Bitbucket would enable automated software quality assessment during collaborative development workflows. Continuous Integration and Continuous Deployment (CI/CD) support would further improve software quality assurance practices.

Advanced Artificial Intelligence models specifically trained for software engineering tasks could improve documentation quality, software understanding, and optimization recommendations. Future research may also explore automated bug detection, vulnerability analysis, technical debt estimation, code clone detection, and automated test case generation.

Additional future enhancements may include:

- Cloud-Based Deployment
- Team Collaboration Features
- Automated Refactoring Suggestions
- Security Vulnerability Detection
- Real-Time Code Monitoring
- AI-Based Bug Prediction
- Intelligent Code Summarization
- Automated Unit Test Generation
- Software Dependency Analysis
- Project-Level Quality Analytics

These enhancements would transform the proposed platform into a comprehensive software engineering and quality management ecosystem.

19 Conclusion

Software quality assurance remains a critical component of modern software engineering because maintainable, readable, and well-documented software systems

significantly influence long-term project success. Traditional software review processes often rely on manual inspection and multiple independent tools, resulting in increased development effort and fragmented quality management workflows.

This research presented an AI-Powered Code Reviewer and Quality Assistant designed to automate software documentation generation, software validation, maintainability assessment, complexity analysis, optimization support, and software quality reporting for Python and Java applications. The proposed platform combines static code analysis techniques with Artificial Intelligence technologies to create a unified software quality environment.

The implementation utilizes Python Abstract Syntax Tree processing for Python source code analysis and Regex-based extraction mechanisms for Java source code understanding. Artificial Intelligence integration through Groq API and LangChain enables intelligent documentation generation, software explanation, and optimization assistance. Additional quality evaluation capabilities are provided through Radon complexity analysis, pydocstyle validation, Streamlit dashboards, and Plotly visualizations.

Experimental evaluation demonstrated that the proposed system successfully improves software documentation quality, maintainability assessment, software readability, and developer productivity. Automated documentation generation reduces manual effort, while validation and complexity analysis mechanisms provide valuable insights into software quality characteristics.

The dashboard analytics module enhances interpretation of software metrics and facilitates informed decision-making during software development activities. Comparative analysis further demonstrated that the proposed platform offers broader functionality than conventional software quality tools by integrating Artificial Intelligence capabilities with software engineering methodologies.

Overall, the AI-Powered Code Reviewer and Quality Assistant demonstrates the potential of combining Artificial Intelligence with software quality analysis to improve software engineering practices. The developed framework contributes toward intelligent automation, improved maintainability, enhanced documentation quality, and efficient software review processes.

Acknowledgment

The author expresses sincere gratitude to Infosys Springboard Virtual Internship 6.0 for providing the learning platform, technical resources, and opportunities that contributed to the successful completion of this project.

The author would also like to thank faculty members, mentors, academic guides, and the open-source software community for their valuable guidance and support throughout the development of the AI-Powered Code Reviewer and Quality Assistant.

The contributions of researchers and developers whose work in Artificial Intelligence, software engineering, software quality assurance, and static code analysis inspired this project are gratefully acknowledged.

References

1. Pressman, R.S., Maxim, B.R.: Software Engineering: A Practitioner's Approach. McGraw-Hill, New York (2019)
2. Sommerville, I.: Software Engineering. Pearson Education, London (2015)
3. Fowler, M.: Refactoring: Improving the Design of Existing Code. Addison-Wesley, Boston (2018)
4. Mens, T., Tourwé, T.: A Survey of Software Refactoring. *IEEE Transactions on Software Engineering* 30(2), 126–139 (2004)
5. Allamanis, M., Barr, E.T., Devanbu, P., Sutton, C.: Machine Learning for Big Code and Naturalness. *ACM Computing Surveys* 51(4), 1–37 (2018)
6. McCabe, T.: A Complexity Measure. *IEEE Transactions on Software Engineering* 2(4), 308–320 (1976)
7. Halstead, M.: Elements of Software Science. Elsevier, New York (1977)
8. Martin, R.C.: Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall (2008)
9. McConnell, S.: Code Complete. Microsoft Press (2004)
10. Spinellis, D.: Code Quality: The Open Source Perspective. Addison-Wesley (2006)
11. Gamma, E., Helm, R., Johnson, R., Vlissides, J.: Design Patterns. Addison-Wesley (1994)
12. Cockburn, A.: Agile Software Development. Addison-Wesley (2002)
13. Beck, K.: Extreme Programming Explained. Addison-Wesley (2005)
14. Humble, J., Farley, D.: Continuous Delivery. Addison-Wesley (2011)
15. Parr, T.: Language Implementation Patterns. Pragmatic Bookshelf (2010)
16. Clements, P. et al.: Documenting Software Architectures. Addison-Wesley (2010)
17. Ayewah, N., Pugh, W.: Using Static Analysis to Find Bugs. *IEEE Software* 25(5), 22–29 (2008)
18. Martin, R.C.: The Clean Coder. Prentice Hall (2011)
19. Python Software Foundation: Python AST Documentation. <https://docs.python.org/3/library/ast.html>
20. Radon Documentation. <https://radon.readthedocs.io>
21. Pydocstyle Documentation. <https://www.pydocstyle.org>
22. LangChain Documentation. <https://python.langchain.com>
23. Groq API Documentation. <https://console.groq.com/docs>
24. Plotly Documentation. <https://plotly.com>
25. Streamlit Documentation. <https://streamlit.io>
26. IEEE Software Engineering Standards. IEEE Computer Society (2023)
27. ACM Digital Library. Software Engineering Research Collection (2024)
28. Booch, G., Rumbaugh, J., Jacobson, I.: The Unified Modeling Language User Guide. Addison-Wesley (2005)
29. O'Reilly: Artificial Intelligence and Software Engineering. O'Reilly Media (2021)
30. Google Developers: Software Engineering Best Practices. Google Publications (2023)